Reducing and Structuring Alive2 Counterexamples for Language-Model-Assisted Repair of LLVM Miscompilations

Abhyuday Vaish
Vellore Institute of Technology Chennai
abhyuday.vaish2024@vitstudent.ac.in

Bhuvan Nayak
Vellore Institute of Technology Chennai
bhuvan.bharatnayak2024@vitstudent.ac.in

Siddharth Sameer Nevgi
Vellore Institute of Technology Chennai
siddharth.sameer2024@vitstudent.ac.in

Tanish Desai
Vellore Institute of Technology Chennai
tanish.somashekhar2024@vitstudent.ac.in

***Abstract*—Alive2 can report a concrete counterexample when an LLVM optimization fails refinement. Its output contains the reproducer, intermediate values, and verifier diagnostics, which a repair model must interpret while editing the compiler's C++ implementation. We study whether reducing this witness and presenting it in a fixed structure help repair. Our feedback layer crosses four reduction choices with two presentation choices, alongside a no-counterexample baseline. The IR-aware reducer edits source and target together, closes candidates over dependencies, uses seeds from the verifier report, and checks accepted changes with Alive2. The planned evaluation uses 24 stratified single-function miscompilations, three trials per bug and condition, and at most four repair turns with Qwen3-Coder-30B-A3B-Instruct. This draft reports an interim snapshot of 205 runs. Seven bugs have all three trials under all nine conditions; across them, raw structured feedback repairs two bugs, IR-aware plain feedback repairs one, and IR-aware structured feedback repairs none. These data do not establish an improvement from IR-aware reduction. The audit also identifies incomplete replacement replies, compilation failures, and unreduced witnesses that limit the comparison. We report these observations separately from a revised harness planned for further evaluation.**

***Index Terms*—compiler testing, translation validation, Alive2, counterexample reduction, program repair, large language models, LLVM**

I. Introduction

When an LLVM middle-end pass miscompiles a function, Alive2 can often demonstrate the error. Given the IR before and after the pass, it searches for an input on which the target does not refine the source, and it reports that input together with the values computed by both sides [1]. The repair loop of *llvm-apr-benchmark* returns this report to the model after a failed patch [2]. The report is verbose relative to the information it conveys. It repeats the reproducer pair (the reproducer alone is 6 to 164 lines in the sample used here) and lists every value computed along the failing execution, most of which are unrelated to the violated refinement.

Two lines of work suggest that the form of failure feedback matters. For the repair of ordinary programs, shortening a large failure-inducing test input before it enters the prompt has been reported to help [3], as has selecting the smallest failing counterexample from a pool [4]. For compilers, recent work studies benchmarks and agent harnesses and reports low resolution rates on LLVM bugs [5], [6]. These settings differ from ours: the feedback here is a *verifier witness for an IR transformation*, while the artifact to repair is the *C++ implementation of the pass*. We ask whether a witness reduced with IR structure and presented in labelled sections helps more than raw feedback. The interim evidence in this draft does not yet establish that benefit.

Within one harness version, the study varies the text supplied to the model from a failed Alive2 check. This includes the initial unpatched failure and later validation failures. Reduction and presentation are separate factors. The benchmark, Alive2, and the underlying repair-loop design are existing work. We contribute the following, described in Sections IV and V:

- a reducer for Alive2 counterexamples that edits source and target in tandem, builds dependency-closed candidates, seeds its search from the verifier's report, and accepts an edit only if an oracle confirms the violation;
- two baseline reducers that differ from it in what they know about the witness, namely line-level delta debugging and a wrapper around `llvm-reduce`, so that prompt length and IR validity can be separated from counterexample awareness;
- a deterministic structured renderer that derives its sections from the IR and the verifier output without a second model; and
- a nine-condition protocol with a prespecified paired analysis whose resolution limits are stated in advance.

The research questions (Section III) concern repair outcomes, repair turns, and reducer cost. We give preliminary outcomes for the completed portion of the sweep and an audit of the harness. The incomplete sweep and subsequent harness revision are kept separate; neither is presented as a completed

positive evaluation.

II. Background and Related Work

A. Refinement, translation validation, and counterexamples

LLVM IR is a typed, SSA-based representation whose semantics distinguish immediate undefined behavior (UB) from special values such as `poison` and `undef` [1], [7]. Because optimizers exploit UB, correctness is defined as refinement rather than equivalence: the target may be more defined than the source or have fewer behaviors, but never less defined [1]. Flags such as `nsw`, `nuw`, and `inbounds` impose additional semantic constraints and can produce poison when their required conditions are violated [7]. Attaching a flag that the source does not justify can therefore break refinement; Alive2's authors report such a case involving `nsw` [1].

Alive2 checks refinement between pairs of functions with an SMT solver. It unrolls loops to a user-chosen bound, so it can miss violations that require more iterations. It is designed to avoid false alarms, including by over-approximating unsupported features and declining to report a violation that depends on such an approximation [1]. It performs the check as a sequence of simpler queries (UB, return domain, poison, undef, value equality, and memory), which supports detailed error messages. Its authors report 47 bugs found by running it over LLVM's unit tests, 28 of which had been fixed at the time of publication [1]. Two properties matter for this work. First, a counterexample is a witness under Alive2's bounded semantics, so a reduction that preserves it preserves only that. Second, the printed report is a by-product of the solver query rather than a document written for a reader.

B. Three artifacts that get reduced, and one that gets repaired

Delta debugging shrinks a failing input by testing subsets and keeping those that still fail. Its `ddmin` variant stops at a 1-minimal input, from which no single element can be removed, and its authors note that knowledge of the input's structure makes it more effective [8]. Compiler testing requires reducers that respect validity. Yang et al. observed that existing delta-debugging variants for C programs introduced undefined behavior, which produced small programs that no longer demonstrated the bug, and they relied on checkers and manual work to avoid this [9]. `llvm-reduce` addresses the same need for IR. It runs delta passes at several granularities against a user-supplied interestingness script and keeps only candidates for which the script exits with status zero [10].

The literature uses "reduction" for three distinct artifacts; we distinguish them explicitly.

1) *Reducing a failing program or input* so that a developer or tool can see the bug. Examples are `ddmin`, `llvm-reduce`, and the reducers that users of Csmith depend on [8]–[10]. The preserved property is that the program still crashes or miscompiles.
2) *Reducing a failing test input before it enters a repair prompt*. ReduceFix has a model generate a `ddmin`-based reducer for each task and accepts a reduction only if the buggy and reference programs still disagree on it. It reports a mean size reduction of 89.1% over successful reductions, on failing inputs whose median size exceeds 1 MB, and pass@10 gains of up to 53.8% relative to a prompt containing the original test. It also finds that shortening the input contributes more than adding output differences [3]. The program under repair is a competitive-programming solution, and the reduced artifact is data that it consumes.
3) *Reducing a verifier witness*, the artifact studied in this paper. The witness is a pair of IR functions plus a report, and the preserved property is a specific refinement violation reported by Alive2. PGS is the nearest relative in the repair literature, but it works differently: it selects the failing counterexample with the fewest input tokens from a pool instead of editing a single counterexample. It reports that minimum-size selection outperformed median and maximum selection, with token count the most effective of three size proxies [4]. Its tasks come from code-generation benchmarks and SWE-bench, not from compiler passes.

What is *repaired* differs again: in this study it is the LLVM C++ that produced the incorrect transformation, not the IR witness.

C. Language-model repair and the form of feedback

Conversational APR alternates patch generation with validation, incorporating earlier patches and validation feedback into subsequent turns rather than repeatedly sampling from the initial prompt [11]. Its feedback consists of test inputs, expected outputs, and error information, and it is evaluated on small programs from the Java and Python versions of QuixBugs. RepairAgent instead lets the model select tools and gather information, rather than following a fixed feedback loop [12]. A survey of 66 LLM-based repair systems classifies them by where the control logic resides: in model parameters, in prompts and context, in designer-specified workflows, or in model-directed control [13]. The loop studied here is a fixed, designer-specified workflow. RepairLLaMA reports that the input and output code representations used for fine-tuning affect repair quality [14]; because we do not fine-tune, this serves only as an analogy. In these systems the feedback consists of tests or properties over source programs. None of the work cited here uses a translation-validation counterexample as feedback or reduces one.

Our model also appears in recent repair studies. PAIR-Bench evaluates Qwen3-Coder-30B-A3B-Instruct on Python submissions using controlled hints about failing behavior [15]. Li et al. use the same model as the starting policy for reinforcement learning on compilation-repair tasks [16]. These studies support studying this model's response to feedback, but they do not supply an expected repair rate for LLVM miscompilations. Our experiment leaves the model weights unchanged and processes the verifier witness rather than training a repair policy or generating hints with another model.

D. Compiler repair benchmarks and harnesses

llvm-apr-benchmark supplies real LLVM middle-end bugs. Each entry has a base commit, reproducer tests, hints, a reference patch, and a build-and-test harness. Miscompilations are checked with Alive2 or, for a few, with `lli` [2]. The benchmark's README reports 295 issues as of May 2025, whereas the copy vendored in our repository contains 491 issue files; we therefore use counts taken from the files (Section VI).

The *llvm-harness* paper builds a 334-bug benchmark and a tool-rich agent harness. It reports that resolution rates decrease by more than 35% (on average 60%) when the evaluated models are tested on its LLVM benchmark rather than on SWE-bench Verified. An expert review finds that a substantial share of benchmark-passing patches are invalid (more than 60% for its best agent–model pair), which puts the true capability of its agents below 21% [5]. LLVM-Bench contributes 423 tasks and an evaluation platform. It reports resolution below 5% for retrieval-based configurations and below 11% for agents, with patch invalidity and build failure as the dominant failure modes [6]. These studies differ in benchmark, model, and protocol, and their results are not comparable to each other or to ours. They matter here for two reasons. They show that the task is difficult enough that a feedback intervention could plausibly matter or plausibly be lost in noise, and they warn that passing a benchmark's checks does not certify a patch. The LLM Compiler models are trained on 546B tokens of LLVM-IR and assembly for optimization and disassembly tasks and are released under a bespoke license [17]. They indicate that models can be trained on IR text, but they are not repair models, and we do not use them.

E. Evaluation practice

Paired binary outcomes on a shared bug set are compared with McNemar's test in the *llvm-harness* paper, which uses a one-sided test with α=0.05 and reports the discordant matrix [5], [18]. A code-generation study adjusts a family of McNemar tests with the Benjamini–Hochberg procedure and reports odds ratios [19], [20]. ReduceFix reports pass@k computed from n sampled patches per bug, treating the samples as drawn without replacement [3]. A robustness benchmark reports that fine-tuned Java repair models lose over 50% of their performance under some semantics-preserving transformations [21]; we cite it only as a reason for caution about identifier changes (Section IX).

III. Problem Formulation and Research Questions

Setting. A bug b consists of a base commit of LLVM, one or more reproducer tests run through `opt`, hints (components, file and line ranges, and the issue title), and a list of regression-test directories. A repair attempt edits a source window W_b of one C++ file. The patched compiler is built and checked by the benchmark's `check_full()` (Section VI-D). If the failing check is an Alive2 refinement failure, the benchmark produces a *witness* $w = (src, tgt, report)$. Here src is the reproducer, tgt is the IR printed by the patched `opt`, and $report$ is the output of `alive-tv`. A feedback function $\varphi_{r,s}(w)$ maps the witness to the text appended to the conversation, where $r \in \{\texttt{raw}, \texttt{generic}, \texttt{llvmreduce}, \texttt{iraware}\}$ is the reduction and $s \in \{\texttt{plain}, \texttt{structured}\}$ is the presentation. The `baseline` condition replaces φ with a fixed sentence stating that the transformation is incorrect. Failures that are not Alive2 refinement failures (build errors, lit failures, and an unmatched patch window) are passed through unchanged under every condition.

Research questions.

- **RQ1 (reducers).** How much do the reducers shrink witnesses, at what verifier cost, and how often do they fail to reduce?
- **RQ2 (reduction).** Does the proportion of bugs repaired differ between `iraware-plain` and each of `generic-plain` and `llvmreduce-plain`? The second comparison separates IR validity from counterexample awareness.
- **RQ3 (presentation).** Does `iraware-structured` repair a different proportion of bugs than `iraware-plain`?
- **RQ4 (anchor).** Does any counterexample help, i.e., does `raw-plain` differ from `baseline`?
- **RQ5 (efficiency).** Do the conditions differ in the number of repair iterations (and thus rebuilds) required, and in verifier calls?
- **RQ6 (generalization).** Does disabling operand promotion change the outcomes of the `iraware` conditions?

The directional hypotheses tested, in the order of RQ2–RQ4, are that `iraware-plain` repairs more bugs than `generic-plain`, that `iraware-plain` repairs more bugs than `llvmreduce-plain`, that `iraware-structured` repairs more bugs than `iraware-plain`, and that `raw-plain` repairs more bugs than `baseline`. These comparisons retain their original directions in the interim analysis.

IV. Proposed Method

Fig. 1 shows where the layer sits in the adapted benchmark loop. The blue boxes are the feedback intervention; the remaining boxes show the repair and validation procedure held fixed within a harness version.

A. The preserved property: the oracle

Every reducer proposes a candidate pair and asks an oracle whether the violation survives. The oracle runs `alive-tv` on the candidate and compares the result with the violation reported for the original pair. That violation is described by the function name, the Alive2 error class (for example, "Target is more poisonous than source"), and a coarse *kind* of target-side outcome (poison, undef, UB, or a concrete wrong value) inferred from the report. Three strictness levels are defined:

- `any_failure`: some refinement failure is still reported, in any function;
- `error_class` (the default): a failure with the same error class is reported in the same function;

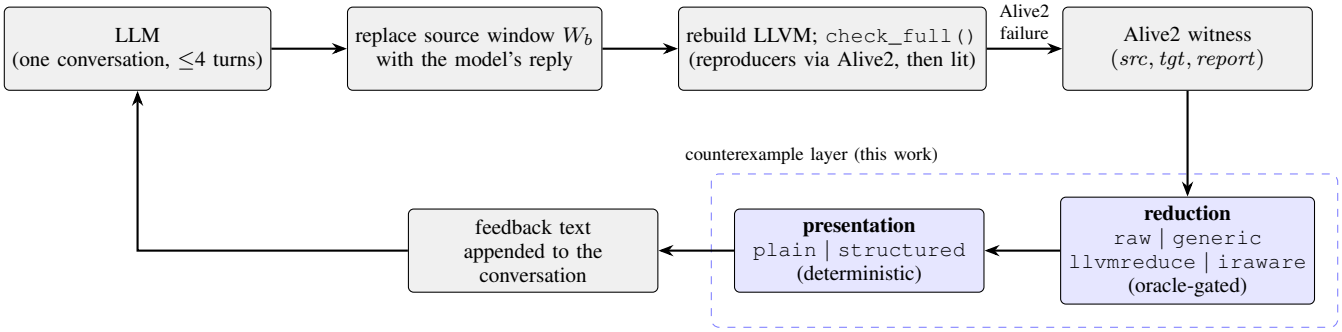


Fig. 1. The counterexample layer (blue) inside the adapted repair loop (grey). Reduction and presentation are separate factors. A passing check ends the run. Other failures (build, lit, unmatched window) bypass the layer. The `baseline` condition substitutes a fixed sentence for an Alive2 counterexample. The initial prompt also includes feedback from the unpatched compiler.

- `error_class_and_kind`: additionally, the target-side outcome kind matches.

A candidate that Alive2 cannot parse or type-check is rejected, as is a candidate whose run errors without reporting a violation. Matching an error class is weaker than preserving the original root cause or the original execution, because two different inputs or two different flags can trigger the same class in the same function. At `any_failure`, the reducer may drift onto an unrelated bug. A reduced witness is therefore a valid counterexample with the same error class in the same function. It is not a proof of equivalence with the original witness, and it is not guaranteed to be globally minimal.

B. Baseline reducers

`generic`. The reduction units are the lines of both files. A variant of `ddmin` proposes subsets and complements, and each candidate is rendered by concatenating the surviving lines of each file. The reducer has no notion that the files are versions of one function, so candidates that contain lines from only one side are simply rejected. Relative to the original algorithm [8], the implementation jumps directly to single-element granularity when no subset or complement succeeds, instead of doubling the number of chunks.

`llvmreduce`. `llvm-reduce` accepts one file and one interestingness test, so the pairing is supplied from outside. The wrapper first reduces the source with the target held fixed. It then reduces the target with the *already reduced* source held fixed, and finally it checks the reduced pair together. The tool's own passes are unmodified. Each candidate spawns a separate process that runs the oracle and returns only an exit status, so the tool never sees the violation or the other file. This isolates "understands IR" from "understands the counterexample."

C. The IR-aware reducer

The reducer operates on a parsed model of the module (Section V) rather than on text. It differs from the baselines in three ways.

Tandem editing. Source and target are two versions of one function. Edits are applied to both and matched by SSA name, so that deleting a value on one side deletes the same-named value on the other.

Dependency-closed candidates. Instead of deleting lines, the reducer chooses a set of values to *keep*. It then closes this set over def-use edges, the operands of terminators and returns, and the branch conditions of every block from which the kept values are reachable. Everything outside the closure is removed. This avoids dangling uses by construction. It does not guarantee a valid module, because type or phi-edge inconsistencies and parser misreads remain possible, which is why every candidate still goes to the oracle.

Counterexample-derived seeds. The verifier report provides three kinds of information. First, it supplies the *seeds*, which are the values the reducer must keep: values whose trace values differ between source and target, values that are poison, undef, or UB on the target side, and values that appear only on the source side. The values returned by the source function are added to this set. Second, it records the *executed blocks* on the source side, which determine the branch to fold. Third, it gives the *type* of each value printed in the example and traces, which is used when a value is promoted to a parameter.

Table I lists the passes, and Algorithm 1 gives the driver. Branch folding is unsound in general (for instance, when the condition is poison), which is one reason the oracle gates it.

Operand promotion generalizes the witness. A fresh parameter may take any value of its type, so the reduced pair may describe an execution that the original program cannot reach. The oracle guarantees that the violation is preserved, but not that it remains relevant to the original bug. Promotion is therefore separately switchable (`--no-promotion`) and is the subject of the ablation in RQ6.

D. Presentation

Plain. The text consists of "Source:" followed by the IR of the (possibly reduced) source, "Target:" followed by the target IR, and then Alive2's own report from the line "Transformation doesn't verify!" onward. The report includes the example input, the per-value traces for both sides, and the final values. When the pair was reduced, the report comes from re-running Alive2 on the reduced pair, so the trace describes the IR that is shown.

Structured. The same IR pair is shown, followed by labelled sections: bug type, verification result, violated property,

TABLE I
PASSES OF THE IR-AWARE REDUCER, IN APPLICATION ORDER. EVERY PROPOSAL IS GATED BY THE ORACLE.

Pass	Proposal
`dce`	delete values that no instruction reads, except seeds (up to 16 sweeps)
`fold-branches`	replace a conditional branch by an unconditional one to the successor that the source trace entered, when exactly one successor was entered
`prune-blocks`	delete blocks unreachable from entry, pruning `phi` edges
`simplify-cfg`	replace single-edge `phi`s by their value; splice a block into its sole unconditional predecessor (up to 32 steps)
`slice`	pin the seeds; `ddmin` over the remaining source values, each subset closed as above and applied to both sides
`promote-operands`	replace a non-seed, non-`phi` value defined on both sides by a fresh parameter of the trace's type, then delete dead producers
`drop-params`	remove parameters unused on both sides
`strip-flags`	`ddmin` over the `nsw`, `nuw`, `exact`, `inbounds`, `nneg`, `disjoint`, `samesign` and fast-math flags on either side

Algorithm 1 IR-aware reduction (as implemented)

Require: src, tgt, parsed violation R (function f), oracle $\mathcal{O}$ with budget B, flag PROMOTE, max_rounds=4

parse both modules; **if** parsing fails or f is missing in either **return** unreduced with an error
$seeds \leftarrow$ diverging, target-poison/undef/UB, and source-only trace values $\cup$ values returned by f_{src}
$types \leftarrow$ value types in R; $exec \leftarrow$ source-side blocks entered in R
$S \leftarrow (f_{src}, f_{tgt})$; $applied \leftarrow \emptyset$
for $round = 1$ **to** max_rounds **do**
  $progress \leftarrow$ **false**
  for each pass P **in** `dce`, `fold-branches`, ..., `strip-flags` **do**
    if $\mathcal{O}$.exhausted **then break**
    $S' \leftarrow P(S)$ ▷ each accepted proposal: $\mathcal{O}(\text{RENDER}(S'))$ preserves the violation
    if $S' \neq \perp$ **then** $S \leftarrow S'$; $applied \leftarrow applied \cup \{P\}$; $progress \leftarrow$ **true**
  if not $progress$ **or** $\mathcal{O}$.exhausted **then break**
if $applied \neq \emptyset$ **and not** $\mathcal{O}(\text{RENDER}(S))$ **then return** the original pair
return RENDER(S)

a diff of source and target keyed by SSA name rather than by line, critical values (the seeds), the dependency chain of the seeds in the source, the blocks entered, the counterexample input, the first divergences between source and target values, memory state when present, and an interpretation. The interpretation is a fixed template per Alive2 error class, filled with values from the report. Six error classes have templates, and other classes receive a one-sentence fallback. A further sentence names any flag that is present in the target and absent in the source. No text is produced by a model. Structured presentation is not information-neutral. It omits the per-value traces that the plain report contains, and it adds derived sections (the diff, the dependency chain, and the interpretation). Differences between `plain` and `structured` are therefore differences in both organization and content.

E. Fallbacks and stopping

Reduction returns the unreduced pair when parsing fails, the named function is absent, or a final check fails. In the evaluated version, exhausting the 400-call reducer budget can itself cause the final check to fail and discard progress. The renderer separately re-verifies a changed pair before using its refreshed trace. That call is outside the reducer's 400-call counter, so this setting is not a bound on all verifier work in a repair run.

The IR-aware reducer stops after at most four rounds of the eight passes, when a round makes no progress, or when the budget is exhausted. It does not claim to reach a fixed point or a globally minimal witness. Within a pass, `ddmin` can reach a 1-minimal subset with respect to that pass's units if it is not interrupted. The `llvm-reduce` wrapper is additionally capped at 600 s of wall-clock time per side. Fallback causes and the size actually shown to the model must be inspected alongside accepted-pass counts.

V. IMPLEMENTATION

A. Evaluated v2 implementation

The implementation is the `ce` Python package and an adapted repair runner in the project repository [22]. The evaluated harness is identified as `repair-v2`; its code is anchored at commit `38b381b`. This draft uses the 205-record data snapshot at `73f2ba1` on branch `feat/h100-sweep`. Older Qwen2.5 runs, earlier Qwen3 runs, and the held-out development pilot are not pooled with it. The benchmark is a vendored copy of *llvm-apr-benchmark* whose upstream commit is not recorded. The repository's build recipe pins Alive2 at `f9a4f02f`; the exact native binary and host package versions still require an environment capture from the H100 node.

Alive2 interface and parser. `alive-tv` is invoked with `--disable-undef-input`, as in the benchmark, after `noalias` and `nofree` are removed from both files (the benchmark's own pre-pass). The parser splits the output at the separator line and reads the error class from the `ERROR:` line. It also reads the example, the two traces (including the `Jump to` annotations), the source and target values, and the memory-state blocks. Alive2's output format is not a documented interface. Anything unrecognized stays in the raw text, and output with no parseable section is reported as a tool error.

IR model. The model is line-oriented and is not an LLVM parser. It separates the module preamble from function definitions and splits each function into parameters, blocks, and instructions, recording result names, opcodes, value operands,

and label operands. Bracketed instructions that span several lines are joined. The repository's parse-and-print check preserves all 1,462 stored reproducer bodies after trailing newline characters are removed for comparison. This checks unchanged round trips, not the correctness of edits. Invalid candidates are normally rejected by the oracle, but error-class matching can still permit semantic drift. Source and target are paired by SSA name, which assumes corresponding values retain their names.

Feedback assembly. After checking the unpatched compiler, the runner processes its failure into the initial user prompt. Later failures are processed in the same way. The adapter selects the first failing test; an Alive2 report receives the selected reduction and presentation, while other diagnostics pass through with an effectively nonbinding 10^9-character cap. In v2, the reduction oracle and trace refresh omit the per-test `additional_args` used by the benchmark's original check. Five sampled bugs carry such arguments: 126012, 114879, 91369, 180870, and 91957. For these bugs, the reduced witness may be checked under different unrolling settings. These records are retained and flagged; a correction cannot retrospectively change the feedback that was shown.

Repair loop. The runner follows the benchmark's hint-based path. The system prompt asks for a small targeted LLVM repair that preserves unrelated optimizations and IR invariants. The first user turn states the bug type, primary component, issue title without its body, condition-specific feedback, and a C++ source window. The window comes from the first hinted file at the base commit, extends the hinted range by 30 lines on each side, and is expanded by a bounded brace-balancing heuristic. This heuristic is not a C++ parser and can still expose partial statements or functions.

In v2, the reply must contain the full replacement window, including requested prefixes and suffixes. Extracted code is substituted for the original window. Before every proposal, the tree is reset to the base commit, so replies are relative to the original source rather than the preceding patch. The conversation retains previous proposals and feedback. Whitespace-normalized unchanged replies consume a turn, receive a fixed corrective message, and bypass building. An unmatched window also consumes a turn without validation. Other replies are built and checked. The v2 extraction and full-window protocol can turn incomplete or poorly formatted replies into invalid C++; these failures remain part of its measured outcome.

Model and serving. We use `Qwen3-Coder-30B-A3B-Instruct`, a sparse model with 30.5B total and 3.3B activated parameters [23]. The operator-reported server uses online FP8 quantization on an H100, vLLM 0.28.0, a 32,768-token maximum context, GPU-memory utilization 0.55, eager execution, and the FlashInfer sampler disabled. The client uses temperature 0.8 and a 300 s request timeout, with no explicit generation seed, `top_p`, or completion-token limit. Effective server defaults and the exact checkpoint revision are not captured by the v2 manifest and remain to be archived. The manifest fixes the model alias, temperature, four-turn budget, oracle settings, prompt hash, and harness identifier. It therefore guards incompatible resumes but is not a complete model or toolchain provenance record.

TABLE II
FROM THE FULL BENCHMARK TO THE SAMPLE, COUNTED FROM THE VENDORED DATASET FILES.

Stage	Bugs
Dataset files (crash 340, miscompilation 142, hang 9)	491
Miscompilations checked with Alive2 (7 are `lli`-only)	135
… with `is_single_func_fix` (eligible subset)	100
… minus bootstrap bug 115575 (sampling pool)	99
Sample: 8 per tier, tiers of 33	24
Interim complete panel (3 trials, all 9 conditions)	7

Version history. Earlier full-window prompts and models produced many unchanged replies. V2 adds a no-op guard, the brace heuristic, and a stronger instruction to make a targeted edit; all nine conditions use these changes. The excluded bug 115575 was used for a nine-condition v2 pilot. It is a development check, not an additional evaluation bug. The original draft's 45 passing and 18 skipped tests describe an earlier source snapshot; those counts do not certify the current native experiment.

B. Revised v3 implementation awaiting evaluation

A revised v3 implementation forwards per-test verifier arguments, uses strict response parsing and atomic search/replace edits, rejects length-truncated completions, and sets an 8,192-token output limit. It also avoids unconsumed terminal feedback, caches previously verified candidate pairs, reserves a final validation call for LLVM-reduce, checks the refreshed failure class, and records the witness actually shown after fallback. These changes affect the edit interface and feedback semantics. They require a fresh output directory and H100 validation on development inputs. No v3 repair outcomes are reported here, and v2 and v3 records will not be pooled. Section VI-G specifies the amended follow-up.

VI. Experimental Design and Evaluation Protocol

The original main protocol specifies 24 bugs, three trials, and nine conditions. This draft audits a frozen partial v2 snapshot and describes a separate v3 follow-up. A repository document dated 2026-09-04 specifies the repeat count, test, and four primary comparisons before the repair records it describes. This is a record in repository history, not an external preregistration. Later model and harness changes are disclosed in a dated amendment. The v3 follow-up is fixed before observing its outcomes, but it is not a completion of the original unchanged protocol.

A. Population

A bug is eligible if it is a miscompilation, every reproducer is checked by Alive2 rather than `lli`, and the reference fix touches one function. The counterexample conditions require the first filter, and the runner skips multi-function fixes. The

TABLE III
THE NINE CONDITIONS. THE BASELINE CONDITION RECEIVES A FIXED SENTENCE AND NO COUNTEREXAMPLE.

Reduction	plain	structured
(none)	baseline	—
raw	raw-plain	raw-structured
generic	generic-plain	generic-structured
llvmreduce	llvmreduce-plain	llvmreduce-structured
iraware	iraware-plain	iraware-structured

pool excludes bug 115575, which was used during harness development. The pool is sorted by hint-region size (the number of lines covered by the bug-location hints), then by reproducer size, then by identifier, and it is cut into three tiers of 33 bugs. Within each tier, bugs are drawn round-robin across the first hinted component, so that InstCombine (45 of the 100 eligible bugs) does not dominate. The procedure is deterministic, and re-running the selection script reproduces the committed list. The tier labels in the repository (easy, medium, hard) describe the size of the hint region and reproducer, not measured difficulty for any model.

The sample has 16 distinct primary component labels, one of which is a placeholder for a bug with no component hint. Eleven of the 24 bugs have knowledge cutoffs in 2025 or 2026. Results will be reported over this subset of miscompilations only, not over the whole benchmark. The runner additionally skips a bug whose failure does not reproduce at its base commit, so the analyzed count may be below 24.

B. Conditions

Held constant. Within v2, every condition uses the same model, prompt template apart from feedback, source window, four-turn limit, oracle budget of 400, `error_class` strictness, and temperature. Feedback length and content differ by design. Other failure types follow the same handling policy, although their diagnostic text varies with each proposal. Earlier Alive2 evidence remains in the conversation during later build or lit failures. LLM sampling is not seeded.

C. Procedure and budget

The main design contains 648 runs: 24 bugs by 9 conditions by 3 trials. The planned promotion-disabled ablation adds 144 runs and is not yet available in this snapshot. A run permits four model replies; unchanged or unapplied replies bypass building, applied replies reach the build stage, and lit runs only after the reproducers pass. The runner uses bug-major order to reuse incremental builds and compiler caches. The reported build concurrency is 64 jobs; compilation and lit run on CPUs rather than the H100 inference device.

Records are written atomically. Resume skips a valid completed run: an early success, or a run that exhausted its reply budget without an endpoint-error flag. Endpoint failure before the first reply leaves the cell empty; later endpoint failures are flagged and treated as incomplete on resume. Failed unpatched builds stop the sweep as setup failures. Final comparisons require the same bugs and all three completed trials per compared condition; missing or infrastructure-interrupted cells are not silently treated as model failures.

D. Success criterion

A run succeeds on the first turn at which the benchmark's `check_full()` returns true. That function performs three steps. (i) It builds LLVM with CMake and Ninja in `RelWithDebInfo` mode with assertions enabled, for the X86, RISCV, AArch64, SystemZ, Hexagon, PowerPC, and NVPTX targets. (ii) It runs every reproducer test through `opt` and requires Alive2 to report zero incorrect transformations, zero failed-to-prove transformations, and zero errors. (iii) It checks out the `llvm/test` tree of the fix commit and runs `llvm-lit` over the directories listed for that bug (one directory for 23 of the 24 sampled bugs and four for bug 114879), stopping at the first failure, with a 300 s limit.

This is a plausibility criterion, bounded by Alive2's unroll limit and by the listed test directories. It is not a proof of correctness and does not cover the whole LLVM regression suite. Because the *llvm-harness* expert review found benchmark-passing patches that were invalid [5], we call a passing patch *plausible*. Manual assessment of the candidate's semantics and optimization preservation remains necessary before describing a harness pass as a confirmed correct repair.

E. Metrics

The primary outcome is the bug-level success indicator *pass@3*: a bug counts as repaired under a condition if at least one of its three completed trials succeeds. Here pass@3 denotes this observed per-bug indicator. When all three trials of a cell exist, it coincides with the unbiased pass@k estimator for $n = k = 3$ [3], but it is not an estimate from a larger sample. *Pass@1*, the fraction of individual runs that succeed, is reported alongside it. One run permits four replies, so pass@1 is not a first-reply success rate. The bug is the unit of analysis: repeated trials do not add independent bugs. Interim repair comparisons use the common panel with all three trials under every condition. The secondary measures are as follows.

- Iterations to repair (LLM turns until success) and the build and verifier-call counts that scale with it. Runs that do not succeed are censored at the iteration limit, so the repository's unconditional mean is not by itself an efficiency estimate. The analysis of this measure is not specified in the repository.
- Reducer measures: instructions, lines, blocks, values, and characters in the witness actually shown; oracle calls; accepted and rejected candidates; tool failures; and seconds. Accepted-pass names alone do not establish size reduction.
- LLM prompt and completion tokens as reported by the server, and an *estimated* feedback size (characters divided by four), which is used only to compare conditions.
- Recorded wall-clock time, which is descriptive. V2 still constructs feedback after the final failed reply, although no model turn consumes it. Per-turn feedback totals omit this terminal work and may undercount oracle activity.

F. Inference

Four comparisons are primary, each in a direction stated beforehand: `iraware-plain` versus `generic-plain`, `iraware-plain` versus `llvmreduce-plain`, `iraware-structured` versus `iraware-plain`, and `raw-plain` versus `baseline`. For a pair, let b be the number of bugs repaired only under the hypothesized-better condition and c the number repaired only under the other. The exact one-sided McNemar test conditions on $n = b + c$ and uses the binomial tail

$$p = \sum_{i=b}^{n} \binom{n}{i} 2^{-n}. \tag{1}$$

The four p-values are adjusted together with the Benjamini–Hochberg procedure [20] at $\alpha = 0.05$. For every comparison, the analysis script also prints the repaired counts, b, c, the paired difference in proportions, an odds ratio, and a two-sided p-value. We report b, c, and the difference regardless of the p-value, and we do not read a non-significant result as evidence of no effect. The other 32 pairs are exploratory and are reported uncorrected and descriptively. The ablation compares each `iraware` condition with itself under promotion on and off, as a separate family of two tests. The script tests the one-sided hypothesis that promotion on is better, so a benefit of disabling promotion would not be detected by that test.

Fewer than five discordant bugs cannot yield $p < 0.05$ one-sided at any split, and $n = 24$ limits power. Under assumed per-attempt rates of 0.12–0.35 and a per-bug difficulty multiplier, the repository's simulation gives power between roughly 0.10 and 0.57 at three trials. The assumptions are the authors' and are based on reported frontier-agent rates rather than on data from this setup. The comparison is therefore best read as a pilot-scale screen. Conditions should have equal numbers of trials before pass@3 is compared.

G. Dated amendment: prospective v3 follow-up

After the v2 audit, an amendment dated 2026-10-22 assigns the original 16 medium- and hard-tier bugs to a separate v3 evaluation. The eight first-tier bugs informed v2 harness debugging and are excluded from that follow-up. The remaining tiers are not a pristine held-out set: bug 115149 was previously used in earlier Qwen3 diagnostics and known-patch validation. The follow-up is prospective with respect to v3 outcomes, not the authors' first exposure to every bug. The source manifest `data/repair_v3_protocol.json` fixes the identifiers, condition set, and budgets before v3 repair outcomes are collected.

To fit the remaining computation budget, the follow-up uses six conditions: `baseline`, `raw-plain`, `generic-plain`, `llvmreduce-plain`, `iraware-plain`, and `iraware-structured`. These retain all four original primary comparisons. Three trials and four replies per run give 16 by 6 by 3 = 288 planned runs. The same generation and validation settings are applied across those conditions, with the v3 changes described above. This subset does not retain the full factorial design, so it cannot estimate every reduction–presentation interaction. No v3 outcome or promotion-off result is available in this draft.

The amendment separates development observations from prospective follow-up data, rather than pooling them into a larger apparent sample. It also reduces the paired sample from 24 to 16; statistical significance is therefore not assured. The hypothesis directions, four-comparison correction family, matched-trial rule, and plausibility criterion remain unchanged. Any secondary-model pilot on the excluded development bug would be a feasibility check and would not contribute to the v3 outcome table.

For v3, candidate-oracle calls and separate trace-refresh calls will be reported individually and together as feedback-verification work. That sum still excludes the benchmark's reproducer and regression-validation work. This avoids treating the 400-call candidate budget as a complete verifier-cost measurement.

TABLE IV
DIAGNOSTIC V2 OUTCOMES ON 7 FULLY MATCHED BUGS (THREE TRIALS EACH). THESE ARE HARNESS-PASSING OUTCOMES, NOT EXPERT-CERTIFIED REPAIRS. INCOMPLETE BUGS ARE EXCLUDED.

Condition	Bugs / 7	Runs / 21
`baseline`	0	0
`raw-plain`	0	0
`raw-structured`	2	4
`generic-plain`	1	1
`generic-structured`	0	0
`llvmreduce-plain`	1	2
`llvmreduce-structured`	1	1
`iraware-plain`	1	1
`iraware-structured`	0	0

TABLE V
DESCRIPTIVE PAIRED COUNTS FOR THE FOUR PRESPECIFIED COMPARISONS ON 7 COMPLETE BUGS. b FAVORS THE HYPOTHESIZED DIRECTION, c OPPOSES IT. P1: IR-AWARE VS GENERIC; P2: IR-AWARE VS LLVM-REDUCE (BOTH PLAIN); P3: IR-AWARE STRUCTURED VS PLAIN; P4: RAW PLAIN VS BASELINE. THIS INTERIM SNAPSHOT IS NOT A SIGNIFICANCE-BASED STOPPING ANALYSIS.

Pair	Both	b	c	Neither	p_{BH}
P1	0	1	1	5	1.000
P2	1	0	0	6	1.000
P3	0	0	1	6	1.000
P4	0	0	0	7	1.000

VII. RESULTS

A. Snapshot coverage

The frozen v2 snapshot contains 205 runs over eight observed bugs. It has ten full-validation passes across three bugs: 89390, 135182, and 126012. The eighth bug, 126012, has not completed all trials and conditions. It is included in the diagnostic totals but excluded from the matched repair table. Seven bugs form the complete panel, with 189 runs and 21 trials per condition. All seven belong to the first size tier, so this panel does not represent the planned 24-bug sample's balance. These are preliminary descriptive results, not a final significance analysis.

TABLE VI
TURN DIAGNOSTICS OVER ALL 205 SAVED RUNS, INCLUDING INCOMPLETE BUGS. CATEGORIES ARE MUTUALLY EXCLUSIVE VALIDATION ENDPOINTS; PARSER AND LINKER SYMPTOMS ARE DIAGNOSTIC SUBSETS, NOT MODEL-CORRECTNESS JUDGMENTS.

Measure	Count
Model turns	810
Applied proposals reaching a build	723
Build-stage failures	467
Reproducer-stage failures	205
Regression-stage failures	41
Full-validation passes	10
Unchanged proposals	87
Build failures with fence contamination	71
Build failures with undefined symbols	124

TABLE VII
INITIAL WITNESS INSTRUCTION COUNTS FROM TRIAL 1 OF EACH PLAIN CONDITION, OVER 8 OBSERVED BUGS. I_0: ORIGINAL; G: GENERIC; L: LLVM-REDUCE; A: IR-AWARE. COUNTS ALONE DO NOT ESTABLISH PRESERVATION OF THE ORIGINAL BUG OR REPAIR USEFULNESS.

Bug	I_0	G	L	A
89390	8	8	3	8
165878	19	19	3	19
135182	10	10	2	10
126974	10	10	10	4
154116	8	8	5	8
116249	21	21	2	18
143123	6	6	4	6
126012	27	27	27	27

B. Matched repair outcomes

Table IV reports the common panel. Raw structured feedback repairs two of seven bugs, while generic plain, LLVM-reduce plain, LLVM-reduce structured, and IR-aware plain each repair one. IR-aware structured, raw plain, generic structured, and the no-counterexample baseline repair none. The successful bugs in this panel are 89390 and 135182. These counts do not show that IR-aware reduction improves repair.

The primary paired counts are given in Table V. IR-aware plain and generic plain each repair a different bug, giving $b = c = 1$. IR-aware plain and LLVM-reduce plain repair the same bug, giving no discordant pairs. The structured IR-aware condition has one loss and no exclusive success relative to IR-aware plain. Raw plain and the baseline both have zero successes. All four BH-adjusted p-values are 1.0. With at most two discordant bugs, these comparisons provide little information about an effect. The larger count for raw structured feedback is exploratory and does not establish superiority.

C. Failures and treatment delivery

Across all 205 records, 810 model replies were recorded. Of these, 87 return unchanged code and 723 reach a build; 467 build attempts fail. Saved diagnostics include stray backticks, undeclared names, missing or incompatible LLVM APIs, and linking errors. A build-stage label alone cannot attribute the failure to hardware or to model reasoning. The source and reply audit identifies failures in the full-window and code-fence protocol, so the compilation rate is partly a property of this harness version. Other failures remain candidate-code errors. Table VI keeps these stages separate.

Ten turns pass full validation and 41 reach regression testing but fail there. Certificate flags record whether any turn passed the reproducer, and are not statements about the final saved patch. Case studies must therefore use the corresponding reply rather than a later failed proposal. Regression failures include optimization expectations in VectorCombine and InstCombine; passing a reproducer alone is not enough.

Offline inspection also limits the interpretation of the ten passing patches. For bug 89390, two LLVM-reduce-condition proposals disable the affected folding step for every integer division or remainder operation. The reference fix instead guards those operations when the shuffle mask contains poison. The proposals may therefore sacrifice valid optimizations even though the selected checks pass; the performance effect has not been measured. We retain their harness-pass labels and do not treat source differences alone as proof of incorrectness. Correctness and optimization-preservation review remain pending.

Table VII summarizes the first initial witness per observed bug. The audit also finds no accepted IR-aware reduction on any recorded witness for 89390, 135182, and 126012. These are the same three bugs with full passes somewhere in the snapshot. Thus a passing repair in an IR-aware condition cannot, by itself, be attributed to successful IR-aware shrinking. An unchanged witness can still receive structured presentation, and stochastic generations can differ even when feedback is identical. The promotion-disabled ablation and the remaining size tiers are not yet evaluated.

VIII. DISCUSSION

The preliminary snapshot shows that the model can produce plausible LLVM repairs, but it does not show an advantage for the proposed reducer. The condition with the most repaired bugs is raw structured feedback, and its advantage is only two bugs in a seven-bug panel. The absence of passes under the no-counterexample baseline and raw plain feedback also leaves the anchor comparison without discordant outcomes. These results should guide an implementation audit, not a new claim of statistical significance.

The main practical limitation is the number of turns that fail before semantic validation. Improving response parsing and the edit interface may recover useful repair attempts, but it would change the harness. Any resulting gain must first be described as a gain from the revised repair procedure. Evidence for the counterexample method would still require comparing its feedback conditions within that revised procedure. Mixing revised runs into v2 would confound the two changes.

Reducer behavior also needs to be separated from repair success. Some initial witnesses are already short, while budget fallbacks or missing verifier arguments can prevent useful edits. The three bugs repaired somewhere in this snapshot have no accepted IR-aware reduction in their recorded feedback. This observation limits attribution, but does not establish that reduction is ineffective on the larger witnesses outside this

v2 snapshot. Evaluating the remaining tiers, inspecting actual shown sizes, and examining fallback causes are needed to address that question.

A useful final analysis should examine whether reduced feedback changes the repairs proposed, whether those repairs preserve existing optimization behavior, and whether any improvement persists without operand promotion. All passing candidate patches should be reviewed before making a claim about correct repairs. Where the evidence remains inconclusive, the paper will report the measured counts and limitations rather than infer equivalence.

IX. Threats to Validity and Limitations

The interim audit exposes implementation and coverage limits in addition to the design limitations below. Their effects cannot be cleanly separated from model capability in the present snapshot.

Witness generalization. Operand promotion can yield witnesses that describe unreachable executions. Only the ablation addresses this, and only through repair outcomes rather than through judgments of relevance.

What the oracle preserves. Equal error class in the same function does not imply equal root cause. The default strictness level was chosen as a compromise, results at other levels are not comparable, and only one level is planned.

Bounded verification. A reduced witness is a counterexample under the same bounded semantics, and the oracle cannot detect violations that require more loop iterations than Alive2's unroll bound. The omitted `additional_args` on five sampled bugs (Section V) is a specific instance of this limitation in the present snapshot.

Parser and model coverage. The IR model is line-oriented, and the round-trip check does not test edits. When a parser error produces invalid IR or destroys the preserved violation, the oracle rejects the candidate, which typically turns the error into a missed reduction. However, because the oracle preserves an error class rather than semantic equivalence to the original witness, undetected semantic drift remains possible. Missed reductions make `iraware` resemble `raw` on the affected bugs. Unreduced fallbacks will be counted in the results.

Reduction failures and budget. A reduction that exhausts the budget is discarded rather than returned, so a tight budget biases the outcome toward unreduced witnesses. The budget of 400 calls is shared by the reducers in different ways: the `llvm-reduce` wrapper counts its subprocess calls against the same budget and additionally has a wall-clock cap that the other reducers lack.

Source and target name matching. Tandem editing and the structured diff assume that names correspond between the reproducer and the IR printed by the patched `opt`. Renumbered unnamed values or renamed instructions would appear as unrelated additions and removals. Only the target function with the reported name is reduced, and other functions in the module are left untouched.

Prompt length and content. Conditions differ in prompt length by construction. The `generic` condition controls for length only if it actually shortens the prompt. The interim size measurements describe only the observed initial witnesses and do not establish the behavior on the remaining sample. In addition, `structured` changes content as well as layout. Length is estimated as characters divided by four.

Treatment reach. Initial Alive2 feedback and later refinement failures are treated. Build and lit diagnostics are passed through without a useful length cap; they can dominate later conversation context. A compilation-failure turn still retains the earlier treatment in its history.

Model dependence and sampling. V2 uses one Qwen model and temperature 0.8. The client does not set `top_p` or an output limit, and the exact checkpoint and server defaults remain to be archived. The effect may depend on model scale and training; three trials do not remove generation noise.

Benchmark contamination. The sampled bugs date from 2024-02 to 2026-02, so a model trained afterward may have seen the fixes. The harness records a self-declared model cutoff date that it does not verify, and we therefore make no claim about contamination-free evaluation. Contamination affects all conditions for a given bug, but it could produce a ceiling at which no condition is discordant.

Localization hints. The window is built from the benchmark's bug-location hints, which the benchmark describes as best-effort and which appear to be derived from the reference fix. The prompt also includes the issue title, which for some sampled bugs names the faulty flag or pass. The experiment therefore measures repair given a localized window, not autonomous localization. The brace-balancing heuristic does not guarantee statement or function boundaries, so a model can receive a partial function even in v2.

Validation scope. The `check_full()` function bounds success as described in Section VI-D, and the Alive2 check passes only on the reproducers that the benchmark lists. A patch can be plausible and still incorrect, as the *llvm-harness* review found in its own setting [5].

Departures from the inherited loop. The runner differs from the benchmark's example loop (Section V). In particular, the `raw-plain` condition does not reproduce the benchmark's prompt exactly, and the resulting rates are not comparable to published rates for the benchmark.

Naming effects. The reducer introduces parameter names of the form `%ce.argN`. We know of no evidence on the effect of such names on models for IR. Evidence from Java repair shows sensitivity to renaming for fine-tuned models [21], but it need not transfer to this setting.

Statistical resolution. With 24 bugs, a non-significant comparison is uninformative about the absence of an effect. Pass@3 and pass@1 can disagree, and iterations to repair are conditional on repair. The contrast of the full method (`iraware-structured`) against `baseline` is not in the primary family and is exploratory.

Scope and interim coverage. The observed complete panel contains only seven first-tier bugs, rather than the planned 24. Findings concern single-function miscompilations checked by Alive2. They do not extend to crashes, hangs, multi-function

fixes, or miscompilations checked by execution. V3 follows a data-informed harness revision and evaluates the original two remaining size tiers. Earlier exposure to bug 115149 is disclosed; prospective collection does not imply an untouched development set. V3 requires separate records and cannot erase the v2 findings.

X. Conclusion

We implemented a feedback layer that reduces and structures Alive2 counterexamples for localized LLVM repair. Its design separates reduction from presentation and preserves a selected failure class through a verifier oracle. The interim v2 evaluation contains 205 runs, but only seven bugs have complete three-trial coverage under every condition. These data show plausible repairs and substantial compilation failures; they do not establish that IR-aware reduction improves repair. The audit motivates a separate harness revision and further evaluation. The final result must depend on matched outcomes and reviewed patches, with the original primary comparisons and all protocol changes reported.

References

[1] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded translation validation for LLVM," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Design Implement. (PLDI)*, Virtual, Canada, Jun. 2021, pp. 65–79, doi: 10.1145/3453483.3454030.

[2] Y. Zheng, "LLVM APR Benchmark: A large-scale automated program repair benchmark of real-world LLVM middle-end bugs," GitHub repository, 2025. [Online]. Available: https://github.com/dtcxzyw/llvm-apr-benchmark

[3] B. Yang, L. Ren, X. Yin, J. Ren, H. Tian, and S. Jin, "Input reduction enhanced LLM-based program repair," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, Rio de Janeiro, Brazil, Apr. 2026, pp. 1109–1121, doi: 10.1145/3744916.3787760.

[4] L. He, Z. Chen, Z. Zhang, X. Gao, and L. Sheng, "PGS: Effective LLM code refinement via property-oriented and structurally minimal feedback," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML), Proc. Mach. Learn. Res. (PMLR)*, vol. 306, 2026, pp. 41 125–41 150.

[5] Y. Zheng, C. Li, S. Li, Y. Zhang, and Z. Su, "Agentic harness for real-world compilers," arXiv:2603.20075 (v2), 2026.

[6] Z. Tian, Y. Zhao, C. Suo, M. Wang, and J. Chen, "LLVM-Bench: Benchmarking and advancing large language models for LLVM compiler issue resolution," arXiv:2607.00700, 2026.

[7] LLVM Project, "LLVM language reference manual," LLVM Documentation, accessed: Oct. 6, 2026. [Online]. Available: https://llvm.org/docs/LangRef.html

[8] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Feb. 2002, doi: 10.1109/32.988498.

[9] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. 32nd ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, San Jose, CA, USA, Jun. 2011, pp. 283–294, doi: 10.1145/1993498.1993532.

[10] LLVM Project, "llvm-reduce – LLVM automatic testcase reducer," LLVM Command Guide, accessed: Oct. 6, 2026. [Online]. Available: https://llvm.org/docs/CommandGuide/llvm-reduce.html

[11] C. S. Xia and L. Zhang, "Conversational automated program repair," arXiv:2301.13246, 2023.

[12] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, Apr. 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[13] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, and H. Tian, "A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications," arXiv:2506.23749 (v3), 2026.

[14] A. Silva, S. Fang, and M. Monperrus, "RepairLLaMA: Efficient representations and fine-tuned adapters for program repair," *IEEE Trans. Softw. Eng.*, vol. 51, no. 8, pp. 2366–2380, Aug. 2025, doi: 10.1109/TSE.2025.3581062.

[15] C. C. Le, A. Yadavally, M. Le-Anh, and T. N. Nguyen, "Benchmarking code improvement with progressive, adaptive, and interactive feedback," arXiv:2607.01360, 2026. [Online]. Available: https://arxiv.org/abs/2607.01360

[16] J. Li, Y. Su, T. Peng, H. Huang, Y. Deng, and M. R. Lyu, "Signal reshaping for GRPO in weak-feedback agentic code repair," arXiv:2605.07276, 2026. [Online]. Available: https://arxiv.org/abs/2605.07276

[17] C. Cummins, V. Seeker, D. Grubisic, B. Rozière, J. Gehring, G. Synnaeve, and H. Leather, "Meta large language model compiler: Foundation models of compiler optimization," arXiv:2407.02524, 2024.

[18] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.

[19] G. Crupi, R. Tufano, and G. Bavota, "Improving code generation via small language model-as-a-judge," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, Rio de Janeiro, Brazil, Apr. 2026, pp. 1238–1249, doi: 10.1145/3744916.3787822.

[20] Y. Benjamini and Y. Hochberg, "Controlling the false discovery rate: A practical and powerful approach to multiple testing," *J. Roy. Statist. Soc. Ser. B (Methodol.)*, vol. 57, no. 1, pp. 289–300, 1995, doi: 10.1111/j.2517-6161.1995.tb02031.x.

[21] F. Rabbi and J. Yang, "HEJ-Robust: A robustness benchmark for LLM-based automated program repair," in *Proc. 3rd ACM Int. Conf. AI-Powered Softw. (AIware)*, Montreal, QC, Canada, Jul. 2026, pp. 371–375, doi: 10.1145/3805760.3814929.

[22] A. Vaish, B. Nayak, S. S. Nevgi, and T. Desai, "better-compiler: IR-aware counterexample reduction toolkit," GitHub repository, branch `feat/h100-sweep`, data snapshot `73f2ba1`, 2026. [Online]. Available: https://github.com/Abhyuday-06/better-compiler

[23] Qwen Team, "Qwen3-Coder-30B-A3B-Instruct: Model card," Hugging Face, accessed: Oct. 22, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3-Coder-30B-A3B-Instruct